



UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ciencias
Escuela Profesional de Ciencias de la Computación

Examen Parcial

Curso: Inteligencia Artificial (CC421)

Semestre 2025-I

Sección 1: Preguntas de opción múltiple [1 punto c/u → 5 puntos]

Instrucciones: Seleccione la(s) respuestas correctas en cada caso.

1. ¿Cuál de los siguientes NO es un tipo de IA según Russell y Norvig?
 - a) Sistemas que actúan racionalmente
 - b) Sistemas que imitan el comportamiento humano
 - c) Sistemas que piensan como máquinas
 - d) Sistemas que piensan racionalmente
 - e) Sistemas que actúan como humanos
2. Durante la ejecución del algoritmo de Búsqueda con Retroceso (*Backtracking Search*) básico para resolver un CSP, cuando se asigna un valor a una variable, ¿cuándo se verifica la consistencia de esta asignación?
 - a) Solo al final, cuando se tiene una asignación completa.
 - b) Después de seleccionar la siguiente variable a asignar.
 - c) Inmediatamente después de asignar el valor a la variable, verificando solo las restricciones que involucran a esa variable recién asignada y otras variables ya asignadas.
 - d) Inmediatamente después de asignar el valor, verificando todas las restricciones que involucran a la variable recién asignada, tanto con variables asignadas como no asignadas.
 - e) En cada paso, verificando todas las restricciones del problema.
3. Dado el 8-puzzle y dos heurísticas h_1 = número de fichas fuera de lugar, h_2 = suma de distancias Manhattan, ¿cuál es cierto?
 - a) Ambas son siempre admisibles y consistentes.
 - b) h_1 domina a h_2 (es decir, $h_2(n) \leq h_1(n)$ para todo n).
 - c) h_2 es más informativa que h_1 , pues produce árboles de búsqueda más pequeños en A^* .
 - d) h_1 puede no ser admisible.
 - e) h_2 no es consistente.
4. En el contexto de la resolución de problemas mediante búsqueda, ¿qué característica se refiere a si un algoritmo garantiza encontrar la solución de menor costo?
 - a) Completitud
 - b) Complejidad en tiempo
 - c) Complejidad en espacio
 - d) Optimización
 - e) Heurística
5. En un juego de suma cero con información perfecta, ¿qué asegura la poda alfa-beta?

- a) Reducir el factor de ramificación sin afectar la decisión óptima.
- b) Encontrar la jugada ganadora en profundidad ilimitada en tiempo polinomial.
- c) Hallar siempre la línea de juego que maximiza el número de estados expandidos.
- d) Convertir el juego en un problema de búsqueda ciega.
- e) Sustituir la función de utilidad por una heurística admisible.

Sección 2: Verdadero/Falso [1 punto c/u → 5 puntos]

Instrucciones: Cada pregunta tiene tres proposiciones. Indique la combinación correcta y justifique brevemente cada una.

1. Respecto a los Problemas de Satisfacción de Restricciones (CSPs):

- ☐ Un CSP se define por un conjunto de variables, dominios para cada variable y un conjunto de restricciones.
- ☐ La búsqueda Backtracking es un algoritmo de búsqueda en profundidad que asigna valores a una variable a la vez y verifica las restricciones de manera incremental.
- ☐ La consistencia de arcos (AC-3) garantiza encontrar una solución si el CSP la tiene.

2. En el contexto de Juegos y el algoritmo Minimax:

- ☐ El algoritmo Minimax explora todo el árbol de juego posible para determinar la jugada óptima.
- ☐ La poda Alfa-Beta es una técnica que permite encontrar la misma decisión que Minimax, pero explorando una menor cantidad de nodos en el árbol.
- ☐ En un juego de suma cero, la ganancia de un jugador es siempre independiente de la pérdida del otro.

3. Considere los algoritmos de búsqueda no informada:

- ☐ La Búsqueda en Profundidad (DFS) siempre encuentra la solución óptima en un espacio de estados con costos de arista uniformes.
- ☐ La Búsqueda de Costo Uniforme (UCS) expande nodos en orden de costo de camino creciente.
- ☐ La Búsqueda Primero en Amplitud (BFS) es completa si el factor de ramificación es finito.

4. En Simulated Annealing:

- ☐ Se aceptan peores soluciones con probabilidad que decrece con la temperatura.
- ☐ Con tasa de enfriamiento demasiado rápida, puede no encontrar el óptimo global.
- ☐ Garantiza encontrar el mínimo global si la temperatura baja de forma logarítmica lenta suficiente.

5. Juegos y Aplicaciones AI - Pac-Man

- ☐ En la versión simplificada de Pac-Man utilizada en las primeras demos del curso, el entorno se considera completamente observado desde la perspectiva de Pac-Man.
- ☐ Si los fantasmas en Pac-Man actuaran de manera completamente aleatoria e independiente del agente Pac-Man, una estrategia basada puramente en MiniMax (asumiendo oponentes adversarios) sería óptima.
- ☐ Un "función de evaluación" (evaluation function) en la búsqueda de juegos MiniMax a profundidad limitada se utiliza para estimar el valor de estados no terminales del juego sin explorar el árbol completo hasta el final.

Sección 3: Pseudocódigo [3 puntos c/u → 6 puntos]

Instrucciones: Resuelva los siguientes problemas proporcionando pseudocódigo claro y conciso.

1. El problema del 15-puzzle (un tablero de 4x4 con 15 fichas y un hueco) es significativamente más complejo que el 8-puzzle. Una parte crucial para la implementación de algoritmos de búsqueda es la generación eficiente de los estados sucesores. Implemente en pseudocódigo una función `GENERAR_SUCESORES_15PUZZLE(estado_actual)` que reciba una representación del estado actual del 15-puzzle y retorne una lista de los estados sucesores válidos. Asuma que el `estado_actual` es una matriz 4x4 donde cada celda contiene el número de la ficha o un símbolo especial para el hueco (ej. 0). La función debe identificar la posición del hueco y generar los estados resultantes de mover las fichas adyacentes al hueco hacia él. No es necesario calcular costos de movimiento en esta función.
2. El algoritmo de Enfriamiento Simulado (*Simulated Annealing*) es una técnica de búsqueda local estocástica útil para problemas de optimización donde se busca minimizar una función de energía. Implemente en pseudocódigo la estructura principal del algoritmo `SIMULATED_ANNEALING(estado_inicial, funcion_energia, funcion_vecino, schedule_temperatura)` que reciba un estado inicial, una función para calcular la energía (costo) de un estado, una función para generar un vecino aleatorio del estado actual, y un esquema para disminuir la temperatura con el tiempo. El algoritmo debe iterar, evaluar vecinos, decidir si aceptar un vecino (incluso si es peor) basándose en la temperatura actual y la diferencia de energía, y actualizar la temperatura según el esquema. El algoritmo debe retornar el mejor estado encontrado durante la búsqueda. Utilice la probabilidad de aceptación $P = e^{-\frac{\Delta E}{T}}$, donde $\Delta E = \text{Energía del Vecino} - \text{Energía del Estado Actual}$ y T es la temperatura.

Sección 4: Pregunta de desarrollo [4 puntos]

Instrucciones: Formalice el siguiente problema como un Problema de Satisfacción de Restricciones (CSP) y describa cómo un algoritmo de resolución de CSPs podría abordarlo.

Considere un pequeño problema de asignación de personal a turnos en una tienda que abre de Lunes a Miércoles. Hay 3 empleados: Ana, Bruno, y Clara. Cada día (Lunes, Martes, Miércoles) requiere exactamente 2 empleados trabajando. Cada empleado tiene restricciones de disponibilidad y preferencias:

- Ana no puede trabajar el Martes.
- Bruno no puede trabajar el Lunes.
- Clara puede trabajar cualquier día.
- Ana y Bruno prefieren no trabajar juntos el mismo día.
- Bruno y Clara deben trabajar juntos al menos un día.

Formalice este problema como un Problema de Satisfacción de Restricciones (CSP) definiendo claramente:

- El conjunto de Variables.
- Los Dominios para cada variable.
- El conjunto de Restricciones, expresándolas de manera clara (puede usar notación formal o descripciones precisas).

Luego, describa cómo la búsqueda Backtracking con Comprobación Hacia Adelante (Forward Checking) abordaría la búsqueda de una solución para este CSP. Explique brevemente qué información propagaría la Comprobación Hacia Adelante después de hacer algunas asignaciones iniciales (ej., asignar empleados para el Lunes).